Part 1: Think Like a Programmer

A computer programmer is a professional who writes, tests, and maintains the code that allows computer software and applications to function correctly.

Code is the written instructions that a computer follows to perform specific tasks.

Unlike instructions for humans which are given in native languages (e.g English, Shona, French), code is instructions written using programming languages.

The instructions are written using programming languages (a language computers understand)

Chapter 1: Introduction to Computing & Programming Fundamentals

Goal: Understand how programming and software development work.

How Computers Work

A computer is an electronic device that accepts data (input), processes it according to instructions, stores information, and produces results (output).

Now this inputting, processing and outputting of data is enabled by software.

Software and Applications

Software refers to programs that instruct a computer on what to do. There are 2 main categories of software.

- **System Software**
    - Software that manages computer hardware.
    - Examples: Operating systems such as Windows, Linux, and macOS.

- **Application Software**
    - Software that helps users perform specific tasks.
    - Examples: Microsoft Word, Chrome, WhatsApp, VLC Media Player.

What is Programming?

Computer programming (aka coding) is the act of writing instructions for computers.

These instructions are written using programming languages.

What is Software Engineering?

Software engineering is the systematic application of engineering principles to design, develop, test, deploy, and maintain software systems.

Software engineering is more than programming

It involves a number of steps which include

- Planning
- Analysis
- Design
- Implementation (where coding comes in)
- Maintenance

Programming Languages

A programming language is a formal language used to communicate instructions to a computer.

Examples of programming languages include Python, Java, C++, JavaScript, C#, PHP

Levels of Languages

- **Low-Level Languages:** Closer to machine code (Assembly).
- **High-Level Languages:** Easier for humans to read and write (Python, Java).

Compiled vs Interpreted Languages

Complied Languages: The entire source code is translated into machine code before execution. **Examples:** C, C++, Go

Advantages

- Faster execution.
- Better performance.

Disadvantages

- Compilation step required.

Interpreted Languages: Code is translated and executed line by line during runtime. **Examples:** Python, JavaScript

Advantages

- Easier debugging.
- Faster development.

Disadvantages

- Generally slower execution.

Python Programming Language

Python is a high-level, interpreted, and general-purpose programming language.

Advantages

- Simple and readable syntax.
- Easy to learn for beginners.
- Large community support.
- Extensive libraries and frameworks.
- Cross-platform compatibility.

Installing Python

- Visit the official Python website:
  - Python.org
- Download the latest version.
- Run the installer.
- Select **"Add Python to PATH"** during installation.
- Verify installation:

python --version

or

python3 --version

Setting Up a Python Development Environment

A Python development environment consists of:

- Python interpreter
- Code editor or IDE
- Required libraries and tools

Basic Setup

- Install Python.
- Install an IDE or editor.
- Create a project folder.
- Write and execute Python programs.

IDEs and Editors

An **IDE (Integrated Development Environment)** provides tools for coding, debugging, testing, and project management.

A **Code Editor** focuses mainly on writing code.

Common Features

- Syntax highlighting
- Auto-completion
- Debugging tools
- Code formatting

Visual Studio Code (VS Code)

Features

- Lightweight and fast.
- Free and open source.
- Supports many programming languages.
- Python extension available.
- Integrated terminal.

Suitable For

- Beginners and professionals.
- Web and software development.

PyCharm

Features

- Specialized for Python development.
- Advanced debugging.
- Intelligent code completion.
- Project management tools.

Suitable For

- Large Python projects.
- Professional software development.

Google Colab

Features

- Cloud-based Python environment.
- No installation required.
- Free access to GPUs and TPUs.
- Ideal for Machine Learning and Data Science.

Suitable For

- Students.
- Researchers.
- Data scientists.

Chapter 2: Key Principles and Building Blocks of Programming

Programming is based on a few fundamental principles that are used in almost every programming language.

↳ 9 cells hidden

Chapter 3: Python Flow Control

Flow control refers to the order in which program statements are executed. Python uses flow control structures to make decisions and repeat tasks.

The main flow control structures are:

1. Conditional Statements - make decisions.
2. Loops - repeat actions.
3. Control Statements - alter loop behavior.

Conditional statements

Conditional statements allow a program to execute different code blocks based on whether a condition is True or False.

```
# if statement
# executes a block of code if a condition is true.
if condition:
  statement
```

```
age = 18
if age >= 18:
    print("You are an adult.")
```

```
# if...else statement
# provides an alternative block for use when the condition is false

if condition:
  statement1
else:
  statement2
```

```
age = 16

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

```
# ladder if statement
# used when there are multiple conditions to evaluate

if condition1:
    statement1
```

```
elif condition2:
    statement2
else:
    statement3
```

```
score = 80

if score >= 75:
    print("Distinction")
elif score >= 65:
    print("Merit")
elif score >= 50:
    print("Pass")
else:
    print("Fail")
```

```
# Nested conditional statements

An if statement placed inside another if statement is called a nested if statement.

if condition1:
    if condition2:
        statement
    else:
        statement
else:
    statement
```

```
age = 20
citizen = True

if age >= 18:
    if citizen:
        print("Eligible to vote")
```

Double-click (or enter) to edit

Loops

Loops allow a block of code to be executed repeatedly. It allows repeating actions

Python provides two main loop types:

1. for loop
2. while loop

```
# for loop
# Used when the number of iterations is known or when iterating through a sequence.

for variable in sequence:
    statements
```

```
for i in range(5):
    print(i)
```

```
fruits = ["Apple", "Banana", "Orange"]

for fruit in fruits:
    print(fruit)
```

```
# while Loop

# Repeats as long as a condition remains True.

while condition:
    statements
```

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Control statements

Loop control statements modify the normal execution of loops.

```
# break

# Terminates the loop immediately.
```

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

```
# continue

# Skips the current iteration and proceeds to the next iteration.
```

```
for i in range(6):
    if i == 3:
        continue
    print(i)
```

```
# pass

# Acts as a placeholder; it does nothing but prevents syntax errors.
```

```
for i in range(5):
    if i == 3:
        pass
    print(i)
```

```
# Loop else

# Python allows an else block with loops.
# The else executes when the loop finishes normally (without a break).
```

```
for i in range(5):
    print(i)
else:
    print("Loop completed")
```

Chapter 4: Python Functions

A function is a reusable block of code that performs a specific task. Functions help break large programs into smaller, manageable, and reusable pieces.

Benefits of Functions

1. Reduce code repetition.
2. Improve readability.
3. Make programs easier to test and maintain.
4. Promote code reusability.

```
# Defining a Function
# In Python, functions are defined using the def keyword.

def function_name():
    statements
```

```
def greet():
    print("Hello, World!")
```

```
# Calling a function
# in calling a function , we simply use the function name

function_name()
```

```
greet()
```

```
# Function Parameters and Arguments

Parameters are variables listed inside the parentheses of a function definition.
They allow data to be passed into a function.

Arguments are values passed into a function when it is called.
```

```
# parameters
def greet(name):
    print("Hello", name)

# argument
greet("John")
```

```
# multiple parameters and arguments
def add(a, b):
    print(a + b)

add(5, 3)
```

```
# Return statement
def add(a, b):
```

```
    return a + b

result = add(5, 3)
print(result)
```

```
# Default parameters
# Parameters with predefined values

def greet(name="Guest"):
    print("Hello", name)

greet()
greet("Alice")
```

```
# Keyword arguments
def student(name, age):
    print(name, age)

student(age=21, name="John")
```

```
# Variable-Length Arguments

Sometimes the number of arguments is unknown.

*args is used to pass a variable number of non-keyword arguments.
**kwargs is used to pass a variable number of keyword arguments.
```

```
# args
def total(*numbers):
    print(sum(numbers))

total(10, 20, 30)
```

```
# kwargs
def display(**details):
    print(details)

display(name="John", age=25)
```

```
# Local Variables

Defined inside a function and accessible only within that function.
```

```
def test():
    x = 10
    print(x)

test()
```

```
# Global Variables

Defined outside a function and accessible throughout the program
```

```
x = 100

def show():
    print(x)

show()
```

```
# The global Keyword

Used when a function needs to modify a global variable.
```

```
count = 0
```

```
def increment():
    global count
    count += 1

increment()
print(count)
```

```
# Docstrings
def add(a, b):
    """Returns the sum of two numbers."""
    return a + b

print(add.__doc__)
```

Chapter 5: Python Data Structures (Built In)

Python provides four commonly used collection data structures: **Lists, Tuples, Sets, and Dictionaries**.

1. Lists

ordered collection of items that can be modified after creation. Lists can store different data types and allow duplicate values.

students = ["John", "Mary", "Peter", "Mary"]

2. Tuples Ordered collection of items that cannot be changed (immutable) after creation.

coordinates = (10, 20)

3. Sets unordered collection of unique items. Duplicate values are automatically removed.

courses = {"Python", "Java", "Python", "C++"}

4. Dictionaries stores data as **key-value pairs**. Each key must be unique.

student = { "name": "John", "age": 21, "programme": "Computer Science" }

Data Structure	Ordered	Changeable	Allows Duplicates
List	Yes	Yes	Yes
Tuple	Yes	No	Yes
Set	No	Yes	No
Dictionary	Yes*	Yes	Keys: No, Values: Yes

Function	Purpose	String Example	List Example	Tuple Example	Set Example	Dict Example
len()	Count items	len("Python")	len([1,2])	len((1,2))	len({1,2})	len({"a":1})
type()	Check type	type("Hi")	type([1])	type((1,))	type({1})	type({"a":1})
max()	Largest item	max("abc")	max([1,5])	max((1,5))	max({1,5})	max({"a":1})
min()	Smallest item	min("abc")	min([1,5])	min((1,5))	min({1,5})	min({"a":1})
sorted()	Return sorted version	sorted("cab")	sorted([3,1])	sorted((3,1))	sorted({3,1})	sorted({"b":2,"a":1})
list()	Convert to list	list("abc")	list([1,2])	list((1,2))	list({1,2})	list({"a":1})
tuple()	Convert to tuple	tuple("abc")	tuple([1,2])	tuple((1,2))	tuple({1,2})	tuple({"a":1})
set()	Convert to set	set("hello")	set([1,1,2])	set((1,1,2))	set({1,2})	set({"a":1})
dict()	Convert to dict	—	dict([("a",1)])	dict((("a",1)))	—	dict({"a":1})
sum()	Add values	—	sum([1,2,3])	sum((1,2,3))	sum({1,2,3})	sum({"a":1}.values())
enumerate()	Index + value	enumerate("abc")	enumerate([1,2])	enumerate((1,2))	enumerate({1,2})	enumerate({"a":1})
zip()	Combine sequences	zip("ab",[1,2])	zip([1],[2])	zip((1,),(2,))	zip({1},{2})	zip({"a"},{"b"})
in	Membership check	"P" in "Python"	1 in [1,2]	1 in (1,2)	1 in {1,2}	"a" in {"a":1}
not in	Membership absence	"x" not in "abc"	5 not in [1,2]	5 not in (1,2)	5 not in {1,2}	"x" not in {"a":1}
count()	Count occurrences	"hello".count("l")	[1,1,2].count(1)	(1,1,2).count(1)	—	—
index()	Find position	"abc".index("b")	[1,2,3].index(2)	(1,2,3).index(2)	—	—
copy()	Create copy	—	[1,2].copy()	—	{1,2}.copy()	{"a":1}.copy()
clear()	Remove everything	—	lst.clear()	—	s.clear()	d.clear()
pop()	Remove item	—	lst.pop()	—	s.pop()	d.pop("a")
remove()	Remove value	—	lst.remove(2)	—	s.remove(2)	—
append()	Add single item	—	lst.append(3)	—	—	—
extend()	Add multiple items	—	lst.extend([3,4])	—	—	—
insert()	Insert item	—	lst.insert(1,99)	—	—	—

Function	Purpose	String Example	List Example	Tuple Example	Set Example	Dict Example
add()	Add item	—	—	—	s.add(5)	—
update()	Update/add values	—	—	—	s.update([3,4])	d.update({"b":2})
keys()	Return keys	—	—	—	—	d.keys()
values()	Return values	—	—	—	—	d.values()
items()	Return key-value pairs	—	—	—	—	d.items()
get()	Safe value retrieval	—	—	—	—	d.get("a")
union()	Combine unique values	—	—	—	a.union(b)	—
intersection()	Common values	—	—	—	a.intersection(b)	—
difference()	Unique to first set	—	—	—	a.difference(b)	—
split()	Break into list	"a b".split()	—	—	—	—
join()	Combine into string	" ".join(["a","b"])	" ".join(lst)	" ".join(t)	" ".join(s)	" ".join(d)
replace()	Replace value	"hi".replace("h","b")	—	—	—	—
strip()	Remove spaces	" hi ".strip()	—	—	—	—
startswith()	Starts with check	"Python".startswith("Py")	—	—	—	—
endswith()	Ends with check	"Python".endswith("on")	—	—	—	—

Chapter 6: Exception Handling

An **exception** is an error that occurs during program execution. When an exception occurs, the normal flow of the program is interrupted.

Exception handling allows programmers to detect and handle errors gracefully without causing the program to crash.

Why Use Exception Handling?

- Prevent program crashes.
- Improve user experience.
- Handle unexpected situations.
- Make programs more robust and reliable.

try: Contains code that may cause an exception except: Contains code that handles exceptions else: Contains code that executes if no exception occurs finally: Contains code that executes regardless of exceptions

Raising Exceptions Python allows programmers to deliberately generate exceptions using `raise`.

age = -5

if age < 0: raise ValueError("Age cannot be negative")

Raising exceptions is important to:

- Enforce business rules.
- Validate user input.
- Detect invalid program states.

User-Defined Exceptions Programmers can create their own exception classes.

class InvalidAgeError(Exception): pass

age = -2

if age < 0: raise InvalidAgeError("Invalid age entered")

Chapter 7: Modules and Packages

As programs grow larger, writing all code in a single file becomes difficult to manage. Python solves this problem through **modules** and **packages**, which help organize code into reusable and manageable components.

1. Python Modules

A **module** is a Python file (`.py`) containing functions, variables, classes, and executable code that can be reused in other programs.

Benefits of Modules

- Code reusability
- Easier maintenance
- Better organization
- Avoid code duplication
- Supports teamwork and collaboration

Creating a Module Create a file named `calculator.py`:

def add(a, b): return a + b

def subtract(a, b): return a - b

Importing and Using a Module

Another Python file can use the module:

import calculator

print(calculator.add(5, 3)) print(calculator.subtract(10, 4))

or importing specific functions

from calculator import add

print(add(5, 3))

Or using aliases (shorter names on importing)

import calculator as calc

print(calc.add(5, 3))

Or alias of the imported function itself.

from calculator import add as sum_numbers

print(sum_numbers(4, 6))

Or importing everything

from calculator import *

Built-in Modules

Python comes with many pre-installed modules.

Module	Purpose
`math`	Mathematical operations
`random`	Generate random numbers
`datetime`	Work with dates and times
`os`	Operating system interaction
`sys`	Access Python system functions
`statistics`	Statistical calculations

Example: import math

print(math.sqrt(25)) print(math.pi)

The `__name__` ***Variable*** Every Python module contains a special variable called `__name__`

2. Python Packages

A **package** is a collection of related modules organized in directories.

Packages help structure large applications.

Package Structure my_package/ | ├── **init**.py ├── math_operations.py ├── string_operations.py

math_operations.py def add(a, b): return a + b

string_operations.py def uppercase(text): return text.upper()

Importing from a Package from my_package.math_operations import add

print(add(2, 3))

Importing Entire Modules from a Package import my_package.math_operations print(my_package.math_operations.add(2, 3))

The `__init__.py` File

The `__init__.py` file indicates that a directory should be treated as a package.

Example: **init**.py print("Package initialized")

Standard Library Packages

Python provides many packages as part of its Standard Library.

Examples:

Package	Purpose
`urllib`	URL handling
`email`	Email processing
`tkinter`	GUI development
`sqlite3`	Database management
`http`	HTTP services

Installing External Packages

External packages can be installed using **pip**.

Syntax pip install package_name

Example pip install requests

Using an Installed Package import requests

response = requests.get("https://example.com") print(response.status_code)

Chapter 8: Engineering Essentials - Linux and Command Line

Learning Linux is one of the most valuable skills for a Python engineer because Python is deeply integrated into the Linux ecosystem. Most professional Python development happens on Linux or Linux-based systems.

For example, you might develop a FastAPI application on your laptop which runs on Windows, but deploy it to an Ubuntu server or a cloud-based Linux virtual machine.

Python developers frequently run commands like:

- python app.py
- pip install requests
- python -m venv venv
- source venv/bin/activate
- pytest These are Linux shell commands. Understanding the terminal makes development much faster.

Most cloud providers use Linux virtual machines. Examples include:

- AWS EC2
- Microsoft Azure Virtual Machines
- Google Compute Engine
- DigitalOcean Droplets
- Oracle Cloud A Python engineer often connects to these servers using SSH

Modern Python applications are commonly containerized using Docker. Docker containers are based on Linux, so understanding Linux makes Docker much easier.

Almost all AI frameworks are developed primarily for Linux:

- TensorFlow
- PyTorch
- Hugging Face Transformers
- CUDA
- JupyterHub servers Training deep learning models is usually done on Linux machines because GPU support is better.

Python engineers often interact with automation tools such as:

- Git
- GitHub Actions Most of these tools are designed to run on Linux.

So basically a Python Engineer should master

Skill	Beginner	Intermediate	Advanced
Navigation (`cd`, `ls`, `pwd`)	✅		
File operations (`cp`, `mv`, `rm`)	✅		
Permissions (`chmod`, `chown`)		✅	
SSH		✅	
Environment variables		✅	
Bash scripting		✅	
Git from terminal		✅	
Process management (`ps`, `top`, `kill`)		✅	
Package managers (`apt`, `yum`)		✅	
System services (`systemctl`)			✅
Cron jobs			✅
Networking tools			✅

1. Introduction to Linux

Linux is a free and open-source operating system based on the Linux kernel. It is widely used for servers, cloud computing, embedded systems, cybersecurity, supercomputers, and software development.

Examples of Linux distributions (distros) include:

- Ubuntu
- Debian
- Fedora
- Rocky Linux
- CentOS
- Arch Linux
- Kali Linux

Linux is:

- Free and open source
- Secure and reliable
- Highly customizable
- Excellent performance
- Supports programming and software development
- Powers most web servers and cloud infrastructure

Linux Architecture +----------------------+ | User | +----------------------+ | Applications | +----------------------+ | Shell | +----------------------+ | Kernel | +----------------------+ | Hardware | +----------------------+

- **Kernel** - Manages hardware resources.
- **Shell** - Interface between the user and the kernel.
- **Applications** - Programs such as Firefox, VS Code, and Python.
- **Hardware** - CPU, RAM, storage devices, and peripherals.

Linux File System

Linux organizes files in a hierarchical tree structure.

Directory	Purpose
`/`	Root directory
`/home`	User home directories
`/root`	Root user's home
`/bin`	Essential executable commands
`/etc`	Configuration files
`/usr`	User applications
`/var`	Variable data and logs
`/tmp`	Temporary files
`/dev`	Device files

Directory	Purpose
`/boot`	Boot loader files

Shell and Command Line

The **shell** is a command-line interpreter that allows users to interact with the Linux operating system.

Popular shells include:

- Bash
- Zsh
- Fish
- Korn Shell (ksh)

Bash is the default shell for most Linux distributions.

Common Linux Commands include:

Common Linux Commands

Command	Description	Example
`pwd`	Display the current working directory	`pwd`
`ls`	List files and directories	`ls`
`ls -l`	List files with detailed information	`ls -l`
`ls -a`	Show all files, including hidden files	`ls -a`
`ls -lh`	Show file sizes in human-readable format	`ls -lh`
`cd directory`	Change to a specified directory	`cd Documents`
`cd ..`	Move up one directory	`cd ..`
`cd ~`	Go to the user's home directory	`cd ~`
`mkdir directory`	Create a new directory	`mkdir Projects`
`rmdir directory`	Remove an empty directory	`rmdir Projects`
`rm file`	Delete a file	`rm notes.txt`

Command	Description	Example
`rm -r directory`	Delete a directory and its contents	`rm -r Projects`
`cp source destination`	Copy a file	`cp report.txt backup.txt`
`cp -r source destination`	Copy a directory recursively	`cp -r Project Backup`
`mv source destination`	Move or rename a file or directory	`mv old.txt new.txt`
`touch file`	Create an empty file	`touch test.txt`
`cat file`	Display the contents of a file	`cat notes.txt`
`less file`	View a file one page at a time	`less largefile.txt`
`head file`	Display the first 10 lines of a file	`head report.txt`
`tail file`	Display the last 10 lines of a file	`tail logfile.log`
`tail -f file`	Monitor a file as it grows	`tail -f server.log`
`nano file`	Edit a file using the Nano editor	`nano notes.txt`
`vim file`	Edit a file using Vim	`vim script.py`
`find path -name "pattern"`	Search for files	`find . -name "*.py"`
`grep "text" file`	Search for text inside a file	`grep "Python" notes.txt`
`clear`	Clear the terminal screen	`clear`
`history`	Display command history	`history`
`man command`	Show the manual page for a command	`man ls`
`echo text`	Display text or variable values	`echo "Hello"`
`whoami`	Display the current username	`whoami`
`hostname`	Display the system hostname	`hostname`
`date`	Display the current date and time	`date`
`cal`	Display a calendar	`cal`
`uname -a`	Show system information	`uname -a`
`df -h`	Show disk usage	`df -h`
`du -sh directory`	Show the size of a directory	`du -sh Downloads`
`free -h`	Display memory usage	`free -h`
`top`	Display running processes in real time	`top`
`htop`	Interactive process viewer (if installed)	`htop`

Command	Description	Example
`ps`	Display running processes	`ps`
`kill PID`	Terminate a process using its PID	`kill 1234`
`killall process`	Kill all processes with a given name	`killall python`
`chmod permissions file`	Change file permissions	`chmod 755 script.sh`
`chown owner:group file`	Change file ownership	`chown user:user file.txt`
`ln -s target link`	Create a symbolic (soft) link	`ln -s file.txt shortcut.txt`
`ssh user@host`	Connect to a remote computer securely	`ssh student@192.168.1.10`
`scp file user@host:path`	Securely copy files between computers	`scp report.pdf user@server:/home/user/`
`ssh-keygen`	Generate an SSH key pair	`ssh-keygen`
`printenv`	Display all environment variables	`printenv`
`env`	Display environment variables	`env`
`echo $VARIABLE`	Display the value of an environment variable	`echo $HOME`
`export VAR=value`	Create or modify an environment variable	`export COURSE="Python"`
`sudo command`	Execute a command with administrator privileges	`sudo apt update`
`apt update`	Update package lists (Debian/Ubuntu)	`sudo apt update`
`apt upgrade`	Upgrade installed packages	`sudo apt upgrade`
`apt install package`	Install a software package	`sudo apt install git`
`apt remove package`	Remove a software package	`sudo apt remove git`
`ping host`	Test network connectivity	`ping google.com`
`curl URL`	Transfer data from or to a server	`curl https://example.com`
`wget URL`	Download a file from the web	`wget https://example.com/file.zip`
`tar -cvf archive.tar folder`	Create a TAR archive	`tar -cvf backup.tar Project/`
`tar -xvf archive.tar`	Extract a TAR archive	`tar -xvf backup.tar`
`zip archive.zip files`	Create a ZIP archive	`zip files.zip *.txt`
`unzip archive.zip`	Extract a ZIP archive	`unzip files.zip`

Bash Basics

Bash (Bourne Again Shell) is the default Linux shell used to execute commands and automate tasks through scripts.

Variables

```
name="Alice"

echo $name
```

Reading User Input

```
read name

echo "Hello $name"
```

Comments

```
# This is a comment
```

Conditional Statement

```
age=20

if [ $age -ge 18 ]
then
    echo "Adult"
else
    echo "Minor"
fi
```

For Loop

```
for i in 1 2 3 4 5
do
    echo $i
done
```

While Loop

```
count=1

while [ $count -le 5 ]
do
    echo $count
    count=$((count+1))
done
```

Creating and Running a Bash Script

1. Create a file.

```
nano hello.sh
```

2. Example script.

```
# !/bin/bash

echo "Hello World"
```

3. Make it executable.

```
chmod +x hello.sh
```

4. Run the script.

```
./hello.sh
```

Linux File Permissions

Linux controls access using permissions assigned to:

- Owner
- Group
- Others

Permission Symbols

Symbol	Meaning
r	Read
w	Write
x	Execute

Viewing Permissions

```
ls -l
```

An example output you would get is:

```
-rwxr-xr--
```

breaking this down you get |owner | group | user | |-rwx | r-x | r-- |

And the meaning of each is

Section	Meaning
rwx	Owner has full access
r-x	Group can read and execute
r--	Others can only read

Changing Permissions

Symbolic mode.

```
chmod u+x script.sh
```

Remove permissions.

```
chmod g-w file.txt
```

Numeric Permissions To make life easy, we can make use of numeric permissions where:

Number	Permission
7	rwx
6	rw-
5	r-x
4	r--
0	---

Example usage then gives

```
chmod 755 script.sh
```

Meaning of this is

```
Owner  : rwx
Group  : r-x
```

```
Others : r-x
```

SSH (Secure Shell)

SSH enables secure remote access to Linux systems.

Connect to a remote computer.

```
ssh username@hostname
```

Example is

```
ssh dzinampini@192.168.1.10
```

Copy Files Securely

```
scp report.pdf user@server:/home/user/
```

Generate SSH Keys

```
ssh-keygen
```

Copy the public key.

```
ssh-copy-id user@server
```

SSH keys allow passwordless login.

Environment Variables

Environment variables store information used by the operating system and applications.

Display All Variables

```
printenv
```

or

```
env
```

Display One Variable

```
echo $HOME
```

Output

```
/home/student
```

Common Environment Variables

Variable	Description
HOME	Home directory
PATH	Search path for executable programs
USER	Current username
SHELL	Current shell
PWD	Present working directory
HOSTNAME	Computer name

Create an Environment Variable

```
export COURSE="Linux"
```

Use it.

```
echo $COURSE
```

PATH Variable

Display PATH.

```
echo $PATH
```

Add a directory.

```
export PATH=$PATH:/home/student/bin
```

Package Management

Install software.

```
sudo apt install git
```

Update packages.

```
sudo apt update
```

Upgrade installed software.

```
sudo apt upgrade
```

Remove software.

```
sudo apt remove git
```

Chapter 9: Engineering Essentials - Version Control using Git and GitHub

What is Version Control?

a system that records changes made to files over time so that you can:

- Return to previous versions
- Track who made changes
- Compare different versions
- Work with multiple developers simultaneously
- Recover deleted work

Think of it as **"Undo" for an entire software project.**

Without Version Control we would have something like:

```
Assignment.doc
Assignment_Final.doc
Assignment_Final2.doc
Assignment_Final_Latest.doc
Assignment_Final_ReallyLatest.doc
```

With version control we have:

```
Version 1
↓
Version 2
↓
```

```
Version 3
↓
Version 4
```

Every version is saved automatically.

2. What is Git?

Git is a **Distributed Version Control System (DVCS)**.

It keeps track of changes made to files on your own computer.

Git works **offline** and stores the complete history of your project.

Git can:

- Track file changes
- Restore previous versions
- Create branches
- Merge code
- Compare versions
- Work without Internet

Git is installed on your computer.

Examples include:

```
My Laptop

Project
│
├── index.html
├── app.js
```

```
├── style.css
└── .git
```

The hidden `.git` folder stores the project's history.

3. What is GitHub?

GitHub is a cloud platform that hosts Git repositories online.

It allows developers to:

- Store repositories online
- Backup code
- Collaborate with teams
- Review code
- Manage projects
- Track issues
- Share open-source software

GitHub requires Git.

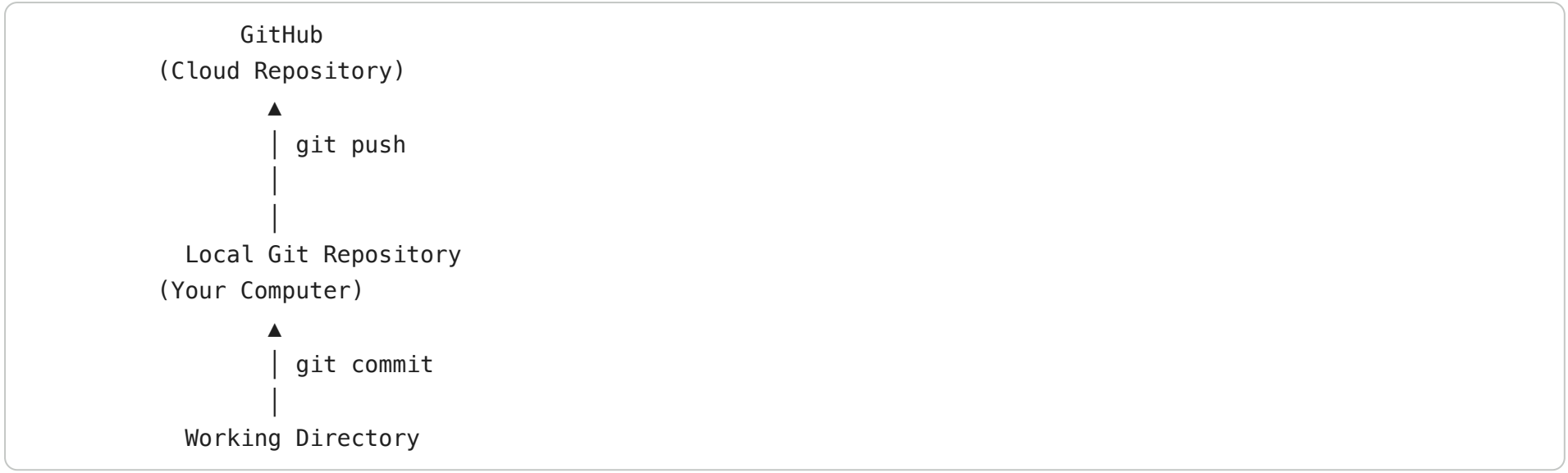
Think of GitHub as **Google Drive for Git repositories**, with collaboration features.

Git vs GitHub

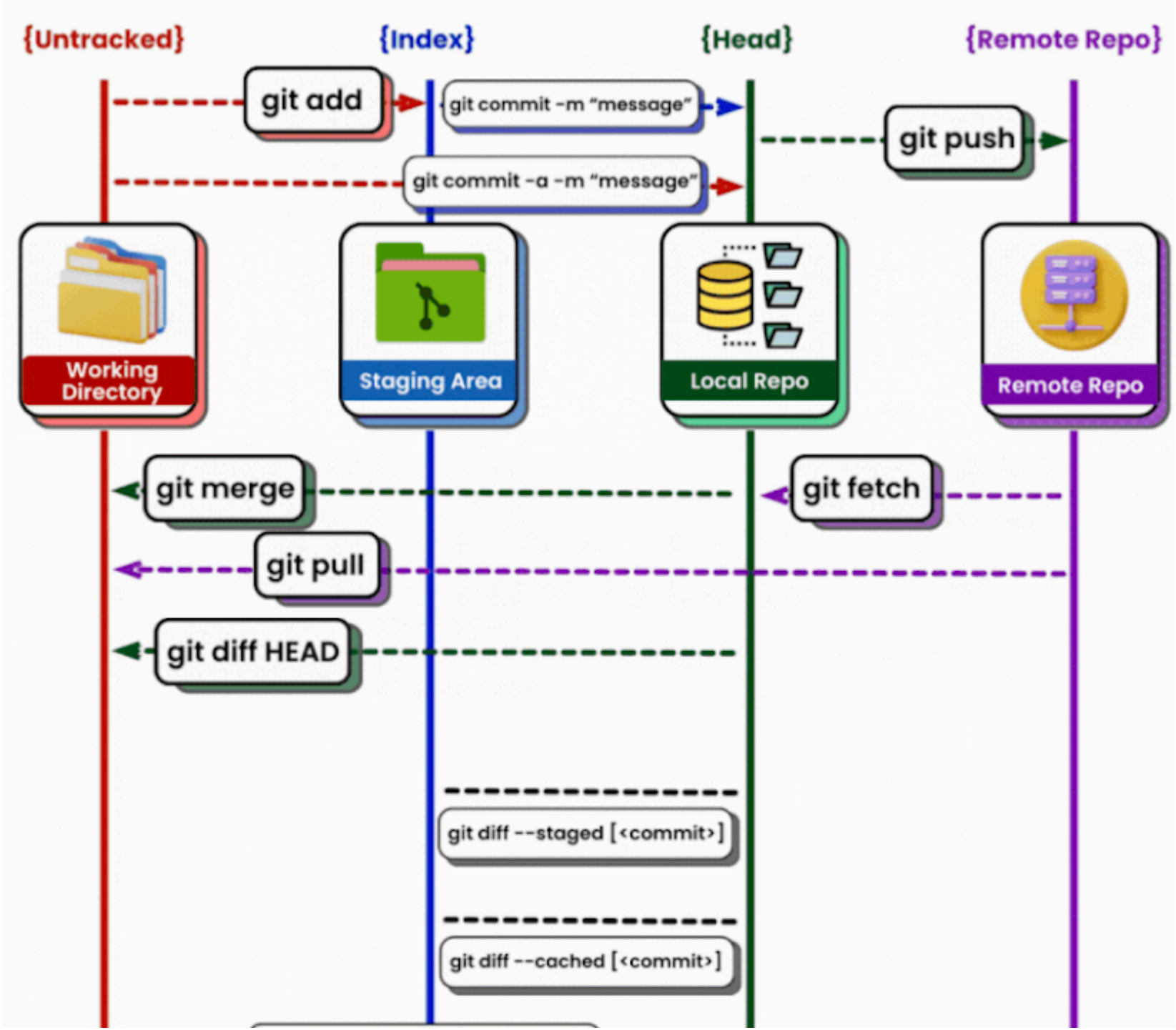
Git	GitHub
Software	Cloud service
Installed on your computer	Runs on the web
Tracks file changes	Stores Git repositories online
Works offline	Requires Internet
Free and open source	Free and paid plans
Handles version control	Handles collaboration

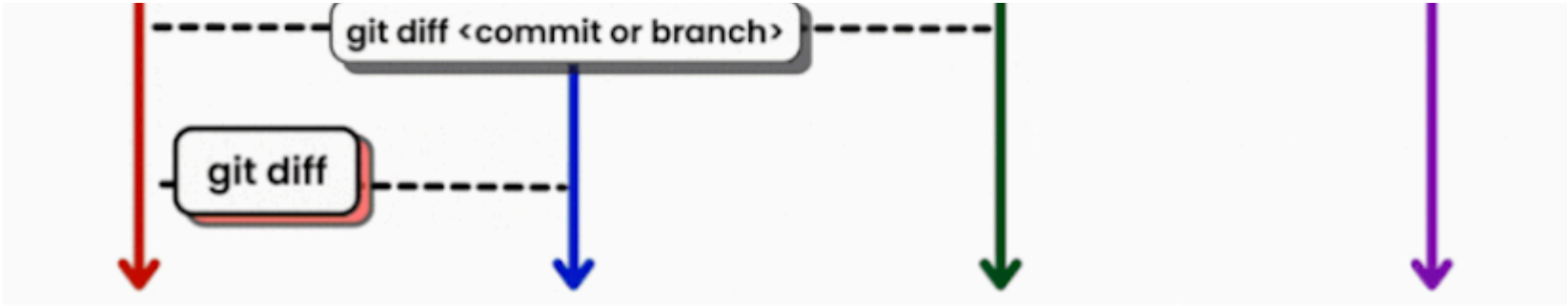
How Git and GitHub Work Together

Think of Git as your notebook and GitHub as the cloud backup.



Git Workflow


{Untracked}
{Index}
{Head}
{Remote Repo}
git add
git commit -m "message"
git push
git commit -a -m "message"
Working Directory
Staging Area
Local Repo
Remote Repo
git merge
git fetch
git pull
git diff HEAD
git diff --staged [<commit>]
git diff --cached [<commit>]



The workflow consists of **four main areas**.

```
Working Directory
        ↓
Staging Area
        ↓
Local Repository
        ↓
Remote Repository
```

Working Directory (Untracked)

This is where you actually edit your files.

Example:

```
calculator.py
README.md
style.css
```

Every time you modify a file, it first appears here.

Example:

```
You edit app.js

↓

Working Directory
```

The files are untracked.

An **untracked file** is a new file Git has never seen before.

Example:

```
notes.txt
```

Git ignores it until you tell Git to track it.

Command:

```
git add notes.txt
```

Staging Area (Index)

The Staging Area is a temporary holding area.

Think of it as a shopping cart.

You choose exactly which files should be included in the next commit.

Example:

```
Working Directory
```

```
app.js
style.css
README.md

↓

git add app.js

↓

This means app.js is now in the staging area (can be included in the next commit)

app.js
```

Only `app.js` will be committed.

To add all you would use:

```
git add .
```

Local Repository and HEAD

The Local Repository stores committed versions.

Every commit becomes a permanent snapshot.

Example:

```
Commit 1
↓

Commit 2
```

```
↓

Commit 3
```

To commit to local repository you use:

```
git commit –m "COMMIT MESSAGE"
```

What is HEAD?

HEAD is a pointer to your current commit.

Example:

```
Commit A

↓

Commit B

↓

Commit C ← HEAD
```

HEAD simply means:

> "This is the version I'm currently working on."

Whenever you make a new commit:

```
Commit D ← HEAD
```

HEAD moves forward.

Remote Repository

The Remote Repository is stored on GitHub.

Example:

```
GitHub

Repository

Commit A

↓

Commit B

↓

Commit C
```

Other developers can access it.

To add the code onto the remote area one uses git push -u origin

Double-click (or enter) to edit

Branching in Git

One of Git's most powerful features is **branching**. It allows developers to work on different features, bug fixes, or experiments **without affecting the main codebase**.

A **branch** is an independent line of development.

Think of it as making a copy of your project where you can make changes safely.

Imagine three developers working on the same project.

- Rudo is adding login.
- Lungani is fixing bugs.
- David is redesigning the homepage.

If everyone edited `main` directly, they could overwrite each other's work or introduce bugs.

Instead:

```
                 login
               /
main ----------
               \
                homepage

               \
                bug-fixes
```

Each developer works independently.

When their work is complete, it is merged back into `main`.

Benefits of Branching

- Multiple developers can work simultaneously.
- Prevents unfinished work from reaching production.
- Makes testing easier.

- Easy to experiment with new ideas.
- Easy to discard unwanted work.
- Enables code reviews through Pull Requests.

Most Common Git Branch Commands

Task	Command
View branches	`git branch`
View all branches	`git branch -a`
Create branch	`git branch feature-name`
Create and switch	`git switch -c feature-name`
Switch branch	`git switch feature-name`
Rename current branch	`git branch -m new-name`
Delete local branch	`git branch -d feature-name`
Force delete branch	`git branch -D feature-name`
Merge a branch	`git merge feature-name`
Push branch	`git push origin feature-name`
Push and track branch	`git push -u origin feature-name`
Delete remote branch	`git push origin --delete feature-name`
Pull latest changes	`git pull`
Fetch remote changes	`git fetch`
Check merged branches	`git branch --merged`
Check unmerged branches	`git branch --no-merged`

Typical Feature Branch Workflow

```
# Start from the latest main branch
git switch main
git pull origin main

# Create a new feature branch
git switch -c feature-user-login
```

```
# Make changes...

# Stage and commit
git add .
git commit -m "Implement user login"

# Push the branch to GitHub
git push -u origin feature-user-login

# Open a Pull Request on GitHub

# After the Pull Request is merged
git switch main
git pull origin main

# Delete the local feature branch
git branch -d feature-user-login

# (Optional) Delete the remote feature branch
git push origin --delete feature-user-login
```

Common Branch Types

Most projects use branches for different purposes.

Branch	Purpose
main	Stable production code
develop	Integration branch
feature	New features
bugfix	Fix bugs
hotfix	Emergency production fixes

Branch	Purpose
release	Prepare a software release

Branching Strategies

A **branching strategy** is a set of rules that tells a development team:

- When to create branches
- How long branches should exist
- When to merge them
- Which branch is considered stable

Different organizations use different strategies depending on:

- Team size
- Project complexity
- Release frequency
- Risk level

The most common strategies are:

1. Git Flow
2. GitHub Flow
3. GitLab Flow
4. Trunk-Based Development

Git Flow

Git Flow is one of the oldest and most structured branching models.

It uses several long-lived branches.

```
                feature/login
               /
develop -------
```

```
                \
                 feature/payment

                 release

main

hotfix
```

Main branches

- `main`
- `develop`

Supporting branches

- feature
- release
- hotfix

The workflow goes like:

1. Create a feature branch from develop.

```
main

develop
   \
    feature/login
```

2. Develop the feature.

```
feature/login

A
↓

B
↓

C
```

3. Merge into develop.

```
develop

------Merge feature------
```

4. When enough features are complete, Create a release branch.

```
develop

↓

release-1.0
```

Only testing and bug fixes happen here.

5. Release branch merges into main.

```
main

Version 1.0
```

6. Merge release back into develop.

This ensures develop has any release fixes.

7. Suppose production crashes, Create a hotfix branch

```
main

↓

hotfix/security
```

8. Fix problem.
9. Merge into

- main
- develop

Advantages

✔ Excellent for large teams

✔ Stable production

✔ Well-organized releases

✔ Multiple versions supported

Disadvantages

✘ Many branches

✘ Complex

✘ Slower releases

GitHub Flow

GitHub Flow is much simpler.

Only one permanent branch exists:

```
main
```

Every task creates a short-lived feature branch.

```
main

↓

feature-login

↓

Pull Request

↓

Review

↓

Merge

↓

Delete branch
```

Each feature is merged independently.

Advantages

✔ Simple

✔ Fast

✔ Perfect for CI/CD

✔ Ideal for startups

Disadvantages

✘ Requires automated testing

✘ Requires frequent deployments

GitLab Flow

GitLab Flow combines Git Flow and GitHub Flow.

Instead of just one production branch, it often has **environment branches**.

Example:

```
main

↓

development

↓

staging

↓
```

```
production
```

Workflow:

```
Feature

↓

Development

↓

Testing

↓

Staging

↓

Production
```

Useful when software passes through multiple deployment environments before reaching users.

Advantages

- Supports deployment pipelines.
- Fits DevOps workflows.
- Easy integration with testing environments.

Disadvantages

- More branches than GitHub Flow.
- Requires good deployment discipline.

Trunk-Based Development

In Trunk-Based Development, everyone works from one central branch called the **trunk** (often `main`).

Branches, if used at all, are very short-lived—typically lasting only a few hours or a day.

```
main

↓

small feature

↓

merge immediately

↓

small feature

↓

merge immediately
```

Instead of large, long-running branches, developers make **small, frequent commits**.

Example

```
Monday

main

↓

Login button

↓

Merge

Tuesday

↓

Password reset

↓

Merge

Wednesday

↓

Dashboard widget

↓

Merge
```

Advantages

- Continuous integration.
- Fewer merge conflicts.
- Fast feedback.
- Ideal for Agile and DevOps.

Disadvantages

- Requires excellent automated testing.
- Developers must integrate changes frequently.
- Large features often need **feature flags** to hide incomplete functionality.

Other Useful Git Commands

git clone

Clones a remote repository on your local computer.

git fetch

Downloads changes from GitHub **without changing your files**.

```
GitHub

↓

git fetch

↓
```

```
Local Repository
```

Think of it as:

> "Download everything but don't apply it yet."

---

git pull

Downloads changes **and merges them automatically**.

```
GitHub

↓

git pull

↓

Working Directory
```

Equivalent to:

```
git fetch
+
git merge
```

---

git merge

Combines two branches.

Example:

```
main

A---B---C

feature

     D---E
```

After merge:

```
A---B---C------F
     \        /
      D------E
```

---

git diff

Shows differences between files.

Example:

```
git diff
```

Shows:

Working Directory vs Staging Area.

---

git diff HEAD

Shows everything changed since the last commit.

```
Current Files

vs

Last Commit
```

---

git diff --cached

Shows staged changes that will be committed.

```
Staging Area

vs

Last Commit
```

---

git diff --staged

Exactly the same as:

```
git diff --cached
```

---

git diff

Compare your current work with an earlier commit.

Example:

```
git diff 9a4f21
```

---

git diff

Compare two branches.

Example:

```
git diff main feature-login
```

Chapter 10: Engineering Essentials 3 - AI-Assisted Coding

AI-assisted coding is **not about replacing software engineers**. It is about **augmenting** their capabilities.

The rise of Large Language Models (LLMs) has changed software development by allowing developers to generate, explain, refactor, test, and document code using natural language.

Instead of spending time writing repetitive code, engineers can focus on solving problems while AI assists with implementation.

AI-Assisted Coding vs Vibe Coding

Vibe coding is when developers simply tell AI what they want and allow it to generate most (or all) of the solution with minimal human oversight.

Characteristics:

- Little planning

- Minimal understanding of generated code
- Accept AI's output without review
- Fast but risky
- Common among beginners

Example:

> "Build me an e-commerce website." AI generates thousands of lines of code that the developer barely understands.

AI-assisted coding on the other hand is a **collaborative approach**.

The engineer remains responsible for:

- Requirements
- Design
- Architecture
- Validation
- Testing
- Deployment

AI helps with implementation. Think of AI as a **junior software engineer** or **pair programmer**, not the technical lead.

Quick Checklist

Remember this simple rule:

> **AI should do the HOW. You should focus on the WHAT and WHY.**

You define:

- the problem
- the requirements
- the constraints
- the architecture

- the business logic

AI helps implement the solution.

Where AI Excels

AI performs exceptionally well on repetitive, structured, and predictable tasks.

- Writing boilerplate code
- Creating project scaffolding
- Writing unit tests
- Writing integration tests
- Generating documentation
- Refactoring existing code
- Debugging syntax errors
- Explaining unfamiliar code
- Converting code between languages
- Creating API clients
- Generating SQL queries
- Creating regex patterns
- Generating Git commit messages
- Writing Dockerfiles
- Creating CI/CD configurations
- Generating configuration files
- Writing comments from code
- Generating code from comments

When NOT to Use AI

AI is powerful, but it should **not** make critical engineering decisions. Avoid relying solely on AI for:

- deciding system architecture

- database selection
- security-critical code
- complex business logic
- learning a concept for the first time
- performance-critical code
- deep domain knowledge

Human-in-the-Loop Principle

One of the most important concepts in AI-assisted coding is the **Human-in-the-Loop (HITL)**.

The AI generates suggestions. The human:

- reviews
- approves
- rejects
- improves
- tests You remain responsible for the final software.

Key Terms

Token A token is the smallest unit of text an AI processes. A token is **not the same as a word**. For example:

```
Engineering
```

may become

```
Engine
ering
```

depending on the tokenizer.

Approximation:

- 1 token ≈ ¾ of a word
- 100 words ≈ 130 tokens

Tokens are used for:

- prompts
- responses
- pricing
- context limits

Context Window The context window is the maximum number of tokens an AI model can "remember" during one conversation.

It includes:

- your prompt
- previous messages
- uploaded files
- generated responses

Example: A model with a 200,000-token context window can work with much larger codebases than one with a 16,000-token limit.

Larger context windows reduce the need to repeatedly provide information.

Prompt A prompt is the instruction you give an AI model. Example:

```
Refactor this Python function using list comprehensions and improve readability.
```

Good prompts produce better results.

Popular AI Coding Tools

Below are some of the most widely used AI-assisted software engineering tools.

Tool	Purpose
GitHub Copilot	AI pair programmer integrated into IDEs
ChatGPT	General-purpose coding assistant
OpenAI Codex	AI coding agent for software tasks
Claude Code	Terminal-based coding assistant by Anthropic
Gemini	Google's AI assistant with coding capabilities
Cursor	AI-first code editor
Windsurf	AI-native IDE focused on autonomous development
Devin	Autonomous software engineer by Cognition
Factory	AI software engineering platform for teams
Jules	Google's asynchronous coding agent
Droid	AI coding assistant focused on Android development
Continue	Open-source AI coding assistant for VS Code and JetBrains
Codeium	Free AI code completion assistant
Tabnine	AI code completion tool
Qodo (formerly CodiumAI)	AI test generation and code quality assistant
Amazon Q Developer	AI assistant for AWS and software development
Replit AI	AI coding assistant integrated into Replit
Sourcegraph Cody	AI code search and coding assistant
JetBrains AI Assistant	AI integrated into JetBrains IDEs
Aider	Terminal-based AI pair programmer using Git
Bolt.new	AI full-stack web application generator
Lovable	AI application builder from natural language
v0	AI UI generator for React and Next.js
Firebase Studio	Google's AI-assisted cloud development environment
PearAI	Open-source AI IDE
Zed AI	AI-assisted collaborative code editor

Powerful Prompting

Good prompts produce better software.

---

1. Plan Before Prompting Don't immediately ask AI to write code.

Instead determine:

- What problem are you solving?
- What are the requirements?
- What constraints exist?
- What files are involved?
- What should success look like?

Think before prompting.

---

2. Establish Rules and Guardrails Tell AI how you want it to work.

Example:

```
Rules

Use Python 3.12
Use snake casing for variable names
```

The better the guardrails, the better the output.

---

3. Treat Every Prompt Like a Sprint Card Each prompt should describe **one well-defined task**.

Use the SMART principle.

A task should be:

- **Specific**
- **Measurable**
- **Achievable**

- **Relevant**
- **Time-bound** (when applicable)

Poor prompt:

> Improve my application.

Better prompt:

> Refactor `UserService.py` to separate validation from business logic. Keep all existing functionality and ensure existing tests continue to pass.

---

4. Provide Context

AI cannot read your mind.

Provide:

- relevant files
- function names
- class names
- error messages
- stack traces
- project structure
- screenshots
- API documentation

Good prompt:

```
Use UserController.py and UserService.py.
Only modify the createUser() method.
Do not change any public API.
```

The more relevant context you provide, the better the response.

---

5. Choose the Right Model Different models have different strengths.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.